

Every verification case ends one of two ways: verified, or *unable to verify*. In the twelve months ending May 2026, UBS worked 876,246 verification searches across four products — employment, education, reference, and license-style checks (EMPV, EDUV, REFV, PLV). Of the 499,465 searches that received at least one attempt, 101,393 ended unable to verify. That UTV rate is the number the business wants down, and it is a frustrating one to manage, because it blends two very different populations: cases that were never going to verify no matter what anyone did, and cases that were winnable but stalled.

The system described here exists to separate those populations and act on the difference. For every open case on every day it answers two questions — *how hard is this case?* and *what is the next sensible move?* — and it answers them

with a strict division of labor: machine-learned models predict, deterministic rules permit. This post explains how the pieces fit, what the evaluation discipline looks like, and the three places we got burned or told ourselves no.

One naming confession up front. The repo is called a verifications *SLM*, and the language-model layer in the system design (P7) is scaffolded — but deliberately unbuilt. What ships today is calibrated tabular models plus a deterministic policy layer. The language layer comes later, if and when the pilot instrumentation can support it honestly. More on that at the end.

The shape of the system

The architecture is five layers, and the arrows matter more than the boxes.

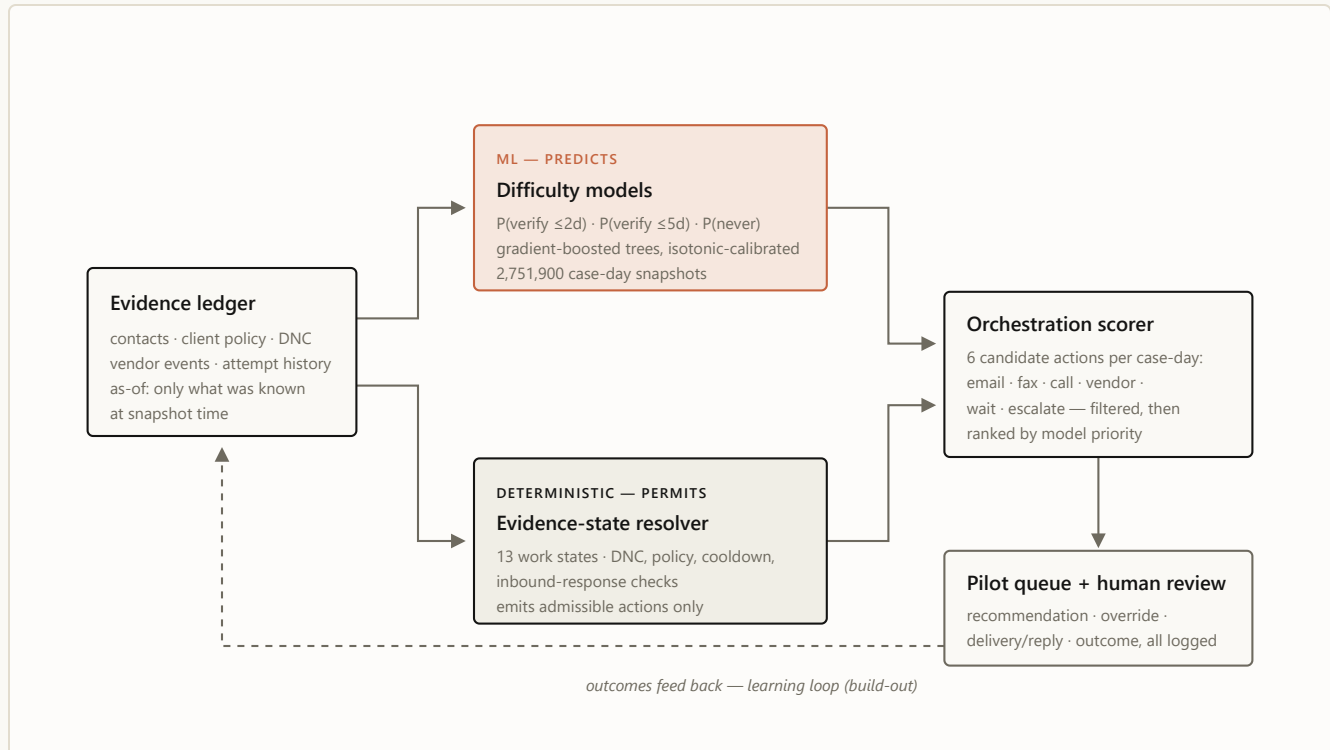


FIGURE 1 The decision path. Models and rules both read the same as-of evidence, but only the resolver decides what is legal. The scorer ranks within legal moves; a human works the queue. The dashed loop — learning channel, cadence, and message from logged outcomes — is the build-out, not the current system.

The load-bearing design rule is the one in the middle:

*Deterministic rules decide what is legal and blocking.
Models only prioritize within legal moves — a
model never authorizes an action it cannot justify.*

Do-not-contact, client policy constraints, third-party exclusions, cooldown timers, and “this person already replied” are all resolved by rules, independent of the models, before any score is consulted. A hard eligibility violation is a filter, not a score penalty a confident model could outvote. In the full 2,751,900-snapshot backtest the resolver produced **zero DNC violations and zero hard-policy violations**, which is only an interesting number because the models never got a chance to create one.

Three questions, one matrix

The modeling layer asks three versions of “how hard is this case?” — will it verify within two days, within five days, and will it *ever* verify. All three are trained from a single matrix of case-day snapshots: one row per `(searchid, snapshot_day)`, 2,751,900 rows over the twelve-month window, each row restricted to what was knowable at that moment. The models are scikit-learn `HistGradientBoostingClassifier`s with isotonic calibration on top, because the consumers of these scores are queues and thresholds, and a

queue built on miscalibrated probabilities is a queue that lies about its own workload.

Table 1. Champion models (v3, promoted 2026-07-02). May 2026 is the 299,317-row holdout gate month; June 2026 time month scored with frozen bundles.

TARGET	CHAMPION VARIANT	BASE RATE	ROC-AUC, MAY	ROC-AUC, JUNE O
Never verifies (final UTV) the headline target	bounded kept — 3 challengers rejected	0.201	0.706	champion held; lift 2.61× vs 2.4
Verifies within 5 days	inbound_text_no_direct promoted	0.477	0.717	0.7
Verifies within 2 days	inbound_text promoted	0.264	0.747	0.7

Two readings of that table matter more than the decimals. First, at day 0 — before any work has happened — a walk-forward, identity-safe variant of the never-verify model reaches **ROC-AUC $\approx$ 0.81** across three forward folds (0.823 / 0.800 / 0.812). Knowing on intake which cases probably won’t come back is the single most useful thing the system does. Second, the pooled model spreads the population hard enough to be operationally meaningful:



FIGURE 2 Observed unable-to-verify rate by predicted-risk decile, May 2026 holdout. The model separates cases that fail 3% of the time from cases that fail 48% of the time. That spread is what makes a *fair* UTV metric possible: misses can finally be measured against what was actually winnable.

That 16× spread is also the business story. Today’s UTV rate silently blames operators for cases that were structurally dead on arrival. Score-conditioned targets flip that: the easy decile is expected to verify almost always, the hard decile almost never, and management attention goes to the winnable middle.

Rules decide what’s legal; models decide what’s next

The deterministic half is an evidence-state resolver that classifies every case-day into one of thirteen work states — terminal, already-responded, policy-blocked, awaiting response, cooldown, review-due, research-needed, digital-paths-exhausted, standard-paths-exhausted, outreach-ready, and so on — and derives the set of admissible actions from the state, never from a score. Here is where the 2.75M snapshots actually sit:

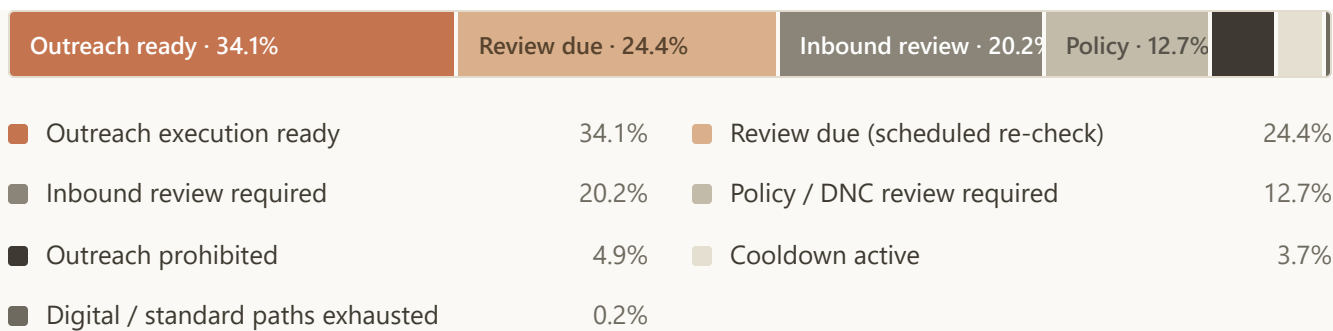


FIGURE 3 Resolver state mix across 2,751,900 snapshots (2026-06-19 ladder run). The interesting segment is the third one: cases where a vendor response is already on record but the legacy flow would have kept contacting.

That third segment hides the most immediately bankable finding in the project. On roughly **401,000 case-days — about one in seven** — a vendor-channel response had already been received, yet the process as practiced would have actively re-contacted the party. The resolver routes every one of them to review-first instead. No model was needed for this; it fell out of writing down, deterministically, what “already responded” means. (The signal is a vendor receipt, not parsed reply content, and it doesn’t exist for every product — the honest version of this number lives in the repo docs alongside its caveats.)

Downstream of the resolver, the scorer builds six candidate actions per case-day — email, fax, call, online vendor, wait, escalate — 16,511,400 candidate rows in the full run, filters them through eligibility, and ranks the survivors by the calibrated scores. A seven-stage pipeline (`run_pipeline.py`) chains candidates → eligibility → scoring → call-escalation ladder → wait policy → workflow router → pilot queue, each stage a subprocess that fails fast on a falsey `passed` in its summary, with a strict contract validator at the end. The full run produced zero blocked rank-1 actions, zero no-action groups, and a queue where every group has an available move.

The 0.93 that wasn't

Early in the project the never-verify model reported ROC-AUC **0.9318**. For a base rate of 0.20 on messy operational data, that number should have set off alarms sooner than it did. It was leakage: the 127-column feature matrix included post-snapshot fields — `thirdpartystatus`, `contactmethod`, lookup-completion fields — plus agent-identity proxies that let the model read the outcome off the case record rather than predict it.

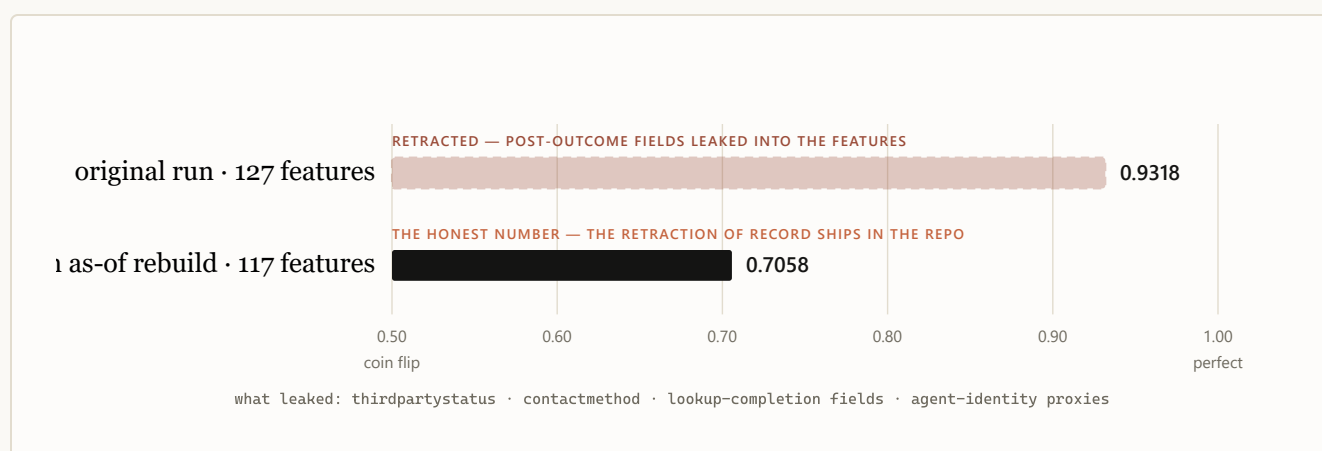


FIGURE 4 The most important chart in the project is the one where the number goes down. The 0.93 read the answer off the case record; the 0.71 predicts. Every number in this post inherits its credibility from this correction — and from the automated audit that now re-checks the boundary on every new matrix.

The fix was a full rebuild under “clean as-of” discipline: 117 columns, every feature re-derived with strictly-before semantics, and an automated leakage audit that now runs against every new matrix. The honest number on the same holdout is **0.706**. We kept the inflated run on disk as evidence with a superseded marker, and checked a retraction of record into the repo itself (`docs/LEAKAGE_RETRACTION.md`) so the correction ships with every clone — anyone who finds the old number also finds the retraction.

Losing 0.23 of AUC hurt for about a day. What it bought is that every number after it is defensible, and the audit that caught it became a permanent gate rather than a one-time cleanup.

Promotion is a gate, not a debate

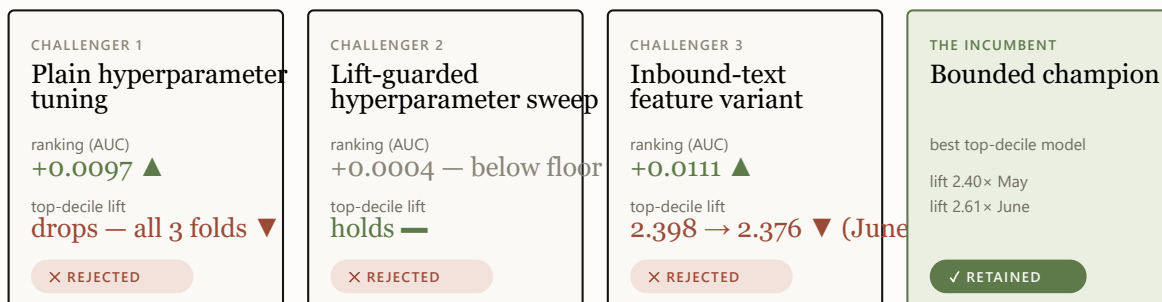
Model improvements are promoted per target — there is no single “champion model,” there are three — and promotion is mechanical. A challenger must clear every criterion against the sitting incumbent;

`config/model_champion_v1.json` is the single source of truth, and rejections are machine-recorded in the champion manifest.

ROC-AUC	at least +0.002 over the incumbent, with bootstrap confidence intervals excluding zero
Brier score	no worse than the incumbent — calibration is not negotiable
Top-decile lift	zero drop tolerated — the top of the queue is what operators actually work
Capture at top 10%	zero drop tolerated
Slice gate	per-product and per-day slices, zero-tolerance with a reviewed, unit-tested allowlist for false positives
Out-of-time replication	the win must survive a never-seen month scored with frozen bundles

The gate earns its keep on the negative results. For the never-verify target, **three consecutive challenger classes have now been rejected on the**

same trade-off: plain hyperparameter tuning (+0.0097 AUC, top-decile drop in all three walk-forward folds), a lift-guarded sweep (held the decile, but +0.0004 AUC missed the 0.002 floor), and this cycle's text-feature variant (+0.0111 AUC, but top-decile lift fell 2.398 → 2.376 on the gate month — and the drop *replicated* on never-seen June, 2.610 → 2.468). Three different levers, one identical failure mode. At that point it stops looking like bad luck and starts looking structural: for this target, average ranking quality and top-decile sharpness are genuinely in tension. The next lever is better labels — June's UTV labels mature in the July pull — and possibly reframing the target itself, not a fourth feature push.



the gate zero-tolerates any top-decile drop — the top of the queue is what operators actually work

FIGURE 5 The gate at work on the never-verify target: three challenger classes, three different levers, one identical failure mode. Each verdict is machine-recorded in the champion manifest — the rejections are as documented as the promotions.

A gate that never says no is a rubber stamp. Ours says no to us regularly, in writing.

Reading the notes without keeping them

The v3 cycle's promoted improvement came from the one data source we had been most careful to avoid: free-text operator and vendor notes on the search history. Notes are where outcome language lives ("verified via..."), so naive inclusion is a leakage machine, and they are also where the privacy risk lives, so persisting them into training artifacts was never on the table.

The compromise is an **11-class deterministic keyword taxonomy that classifies each note in flight during export and then throws the text away**. What lands on disk per event is the class flags and a SHA-256 pointer — every partition manifest records `raw_note_text_persisted: false`. From 17,896,639 classified events across 871,844 searches, the matrix gains 30 as-of features per snapshot (per-class counts, last-flags, recency, note size), computed with strictly-before semantics and boundary unit tests. About 28% of snapshots carry at least one classified text event at their as-of time.

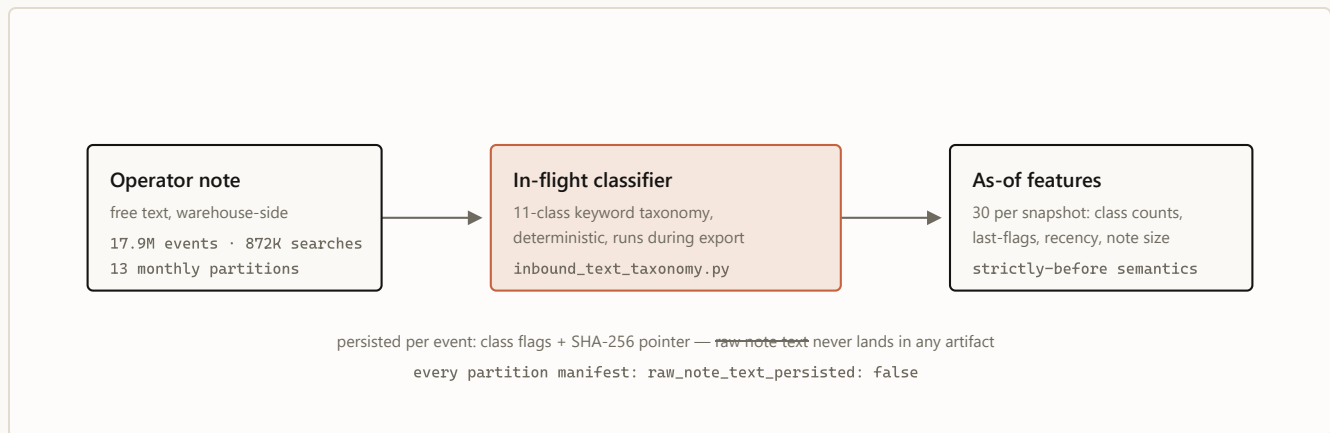


FIGURE 6 The inbound text layer. Signal crosses the boundary; text does not. The one taxonomy class that never fired (`undeliverable`: 60 hits in 17.9M events) shipped as a dead feature and is first in line for the v2 phrasing pass.

The features cleared the full gate for both success targets. The five-day model improved on every metric with bootstrap CIs excluding zero, and — the part we watch most closely — the gain *grew* out of time: +0.0065 AUC on the gate month, +0.0071 on never-seen June. The two-day model held the same pattern. The never-verify variant, as covered above, gained AUC, lost the top decile, and died at the gate. One near-miss worth recording: the slice gate’s name scanner flagged `text_redirect_to_third_party_*` as a possible label proxy — a pattern collision, since the guarded pattern protects post-outcome completion columns. The features passed the timestamp audit, were reviewed and allowlisted, and a unit test now proves that *unreviewed* pattern matches still fail. The scanner being slightly paranoid is the correct setting.

INCIDENT REPORT — TWO EXPORTERS, ONE LOCK

During the v3 export, two `--skip-existing` relaunches raced: an exporter from before a session cutoff had survived and kept writing. Interleaved truncate/append left partitions at $\sim 1.8\times$ their true row count with a single header — subtly plausible files, exactly the kind that poison everything downstream.

The manifest saved us. A row-count cross-check against each partition’s manifest failed *before any consumer read the data*; all parts were wiped and re-exported under a new exclusive pidfile lock, with Windows-aware liveness checking (on Windows, `os.kill(pid, 0)` terminates the target — `OpenProcess` it is) and three regression tests. Cheap invariant checks between pipeline stages are the highest-ROI code in this repo.

What we deliberately don't claim

The repo maintains an explicit disallowed-claims list, and it does more for the project's credibility than any AUC. The current lines:

- **No causal claims about channel, wording, or templates.** Historical patterns like “email-first cases verified more often than call-first cases” are observational — selection effects all the way down. Causal language stays banned until the pilot logs the full chain for each case: recommendation shown, action taken or override reason, template version, delivery/reply, final outcome. The logging schema for exactly those five event types is built and validated; the evidence is not yet collected.
- **Not “identity-free,” only “direct identifiers stripped.”** Action-facing models exclude direct agent and team IDs behind a drift gate, but behavioral agent attributes remain. Identity-bearing champions stay `provisional` for action-facing use, and the resolver applies no-direct-identity fallbacks automatically until that gate approves.
- **Decision support only.** No auto-send, no automated final decisions, exploration disabled by default. The router is shadow-review metadata, not an autonomous policy — and we won't train a “learned policy” on historical action logs and pretend the logs were experiments.
- **No language model yet.** The endgame — a small language model that drafts the emails, faxes, notes, and call scripts, learning from past communication to draft the best next action — is sequenced last on purpose, because without the causal instrumentation above we couldn't tell whether it works.

What's next

Three threads, in order of readiness. The **July cycle** re-runs the monthly spine — the text exporter and matrix builders are month-partitioned and resumable for exactly this — and matures June's never-verify labels, which is the most promising lever left for the one target that keeps rejecting challengers. The **identity proxy gate** either approves the identity-bearing champions for action-facing use or keeps the fallbacks in place; the walk-forward drift harness exists, and the decision is deliberately not ours to hand-wave. And the **play engine** — recommending channel + timing + message family + template as a unit, then eventually learning that choice from propensity-logged pilot data — waits on the only thing that can unlock it: real logged outcomes from humans working the queue.

And after the play engine, the layer this repo is named for. The final step is a **small language model that writes the communications themselves** — the emails, the faxes, the case notes, and the scripts our call operators work from — learning from the logged record of past communication and its outcomes to draft the best next action for every case. It is sequenced last because it is graded last: only the pilot's causal instrumentation can tell a well-worded email from a lucky one.

The system today is, on purpose, the boring version: calibrated probabilities, deterministic guardrails, mechanical promotion gates, and a written list of things we refuse to say. Every flashier layer gets built on top of that — or not at all. ♦

Reproducing this. From the repo root: `py -3.12 code\validate_repo.py` (canonical gate: compile, tests, configs) · `py -3.12 code\run_pipeline.py` (seven-stage decision path) · `py -3.12 code\validate_decision_contract.py strict` · `py -3.12 code\pilot_logging.py validate`.

The series. This deep dive is part 1. Also: [Earned autonomy](#) (the design philosophy) · [the illustrated walkthrough](#) (one case through the machine) · [the executive briefing](#) · [the Murphy Brown field report](#).

Internal engineering note written in the style of a public post. All figures are backtested and as-of; nothing here claims pilot results or causal channel/template effects. Sources of record in-repo: `README.md`,

`docs/LEAKAGE_RETRACTION.md`, `docs/TUNED_CHAMPION_PROMOTION_20260702.md`,

`docs/INBOUND_TEXT_PROMOTION_20260702.md`, `docs/JUNE_2026_OOT_VALIDATION.md`,

`config/model_champion_v1.json`, `presentations/`.

UBS Verifications · Decision Engine · July 2026